



SYSTEM DESIGN & TECHNICAL OVERVIEW

# Kali Tools GUI

מסמך תכן והסבר מערכת — ארכיטקטורה, סוכנים וזרימת נתונים

מחבר: **הראלי דודאי**

ת.ז.: 027225721

דוא"ל: hareli26@gmail.com

מוסד: **מכללת HackerU** · קורס: אדריכלות AI

מנחה: שחף

תאריך: יולי 2026

מאגר קוד: [github.com/Hareli26/kali-tools-gui](https://github.com/Hareli26/kali-tools-gui)

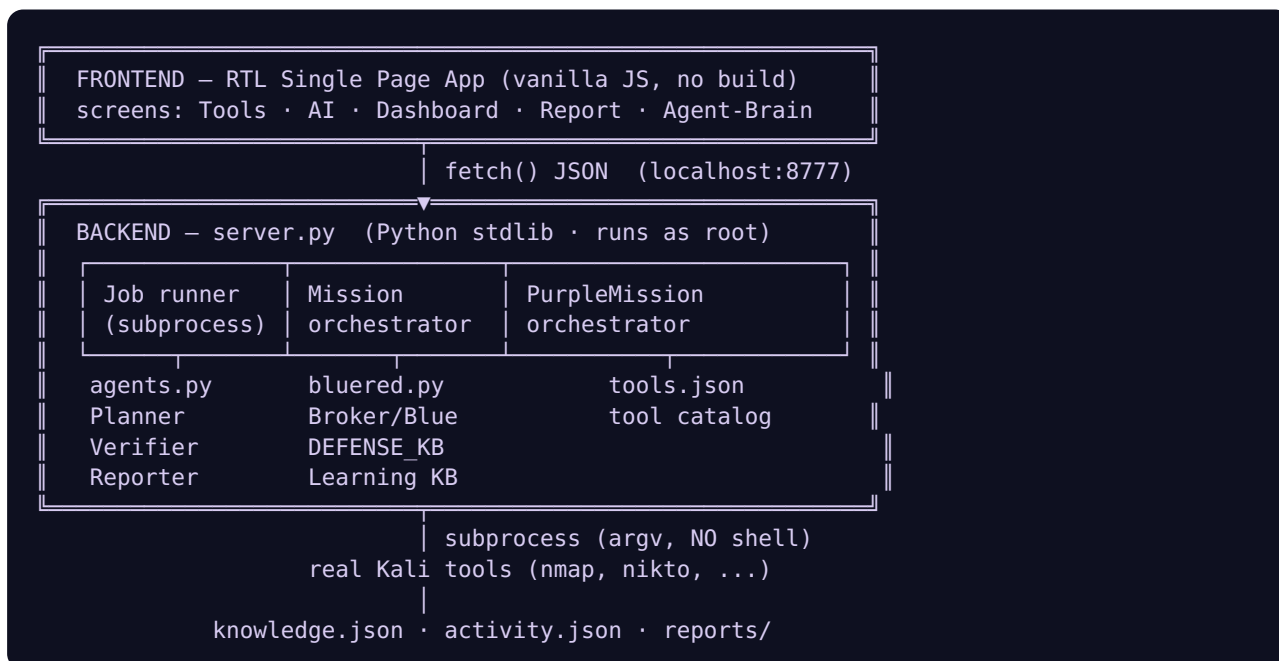
## תוכן עניינים

1. מבוא
2. סקירת ארכיטקטורה כללית
3. שכבת ה-Backend (server.py)
4. מנוע הכלים ובניית פקודות
5. שכבת הסוכנים (agents.py)
6. שכבת Purple Team (bluered.py)
7. זרימת נתונים — מאיפה הסוכנים יודעים?
8. שכבת ה-Frontend ותצוגת המוח
9. אבטחה
10. פריסה ותפעול בייצור
11. טכנולוגיות (Tech Stack)
12. צילומי מסך מהמערכת
13. סיכום

## 1. מבוא

מסמך זה מתאר את התכן ההנדסי של מערכת **Kali Tools GUI**: ממשק גרפי ושכבת סוכנים בגישת Purple Team המורצת מעל כלי Kali Linux תחת WSL2. המסמך נועד להסביר כיצד המערכת בנויה, כיצד רכיביה משתלבים, ובמיוחד — מאיפה הסוכנים "יודעים" לפעול ולקבל נתונים.

## 2. סקירת ארכיטקטורה כללית



איור 1 — שכבות המערכת וזרימת הנתונים.

## 3. שכבת ה-Backend (server.py)

שרת HTTP רב-תהליכי (ThreadingHTTPServer) בספריית פייתון הסטנדרטית. מגיש את קבצי ה-Frontend וחושף API ב-JSON.

### 3.1 מודל ה-Job

כל הרצת כלי היא אובייקט Job: מריץ subprocess.Popen בקבוצת תהליכים נפרדת, קורא פלט בתהליכון (thread) עם מגבלת גודל, ומאפשר עצירה נקייה. הפלט זמין ל-polling מהדפדפן.

### 3.2 נקודות קצה עיקריות

| תפקיד                                      | נקודת קצה          |
|--------------------------------------------|--------------------|
| קטלוג הכלים + סטטוס התקנה                  | GET /api/tools     |
| הרצת כלי בודד                              | POST /api/run      |
| Planner: כוונה+מטרה → תוכנית שלבים         | POST /api/plan     |
| הרצת משימה (Executor+Verifier+Reporter)    | POST /api/mission  |
| הרצת Purple (Red→Broker→Blue→Orchestrator) | POST /api/purple   |
| מצב סוכנים חי + מודיעין + יומן             | GET /api/dashboard |
| גרף יכולות הסוכן + נתונים חיים             | GET /api/brain     |
| בריאות השירות (גרסה, uptime)               | GET /api/health    |

## 4. מנוע הכלים ובניית פקודות

הקטלוג tools.json מגדיר לכל כלי: בינארי, קטגוריה, ואת שדות הטופס (סוג, דגל, ברירת מחדל). פונקציית build\_argv בונה את הפקודה כמעריך argv — כל אסימון בנפרד, ללא shell. זהו עיקרון אבטחה מרכזי: ערכי המשתמש לעולם אינם מתפרשים ע"י מעטפת, ולכן הזרקת פקודות נמנעת.

```
argv = [binary]
for opt in options:
    # לפי סדר ההגדרה
    if bool ומסומן: argv += [flag]
    elif ערך קיים: argv += [flag, value] # או "flag=value"
argv += [positional target]
```

## 5. שכבת הסוכנים (agents.py)

מנוע מבוסס-חוקים (Playbooks) — ללא תלות ב-LLM, דטרמיניסטי וניתן לביקורת.

### Planner (מתכנן) 🗺️

מקבל כוונה חופשית + מטרה. מנקד כל Playbook לפי מילות מפתח (עברית+אנגלית), בוחר את המתאים ובונה שלבים (כלי + פקודה + הסבר + הצעה). לדוגמה: "בדיקת dig/dnsenum → DNS" /DNSrecon.

### Executor (מבצע) ▶️

מריץ כל שלב ברצף כ-Job, עם מגבלת זמן לכל שלב (ברירת מחדל 300 שניות) כדי שכלי איטי לא יתקע את המשימה.

### Verifier (מאמת) 🔍

מנתח את הפלט וקוד היציאה של הכלי. באמצעות דפוסי regex מזהה שגיאות (דרוש root, מטרה לא זמינה) ומחלץ ממצאים (פורטים פתוחים, CVE, חולשות). מפיק verdict: תקין / ממצאים / אזהרה / נכשל.

### Reporter (מדווח) 📄

מרכיב דוח Markdown: סיכום מנהלים, ממצאים עיקריים ופירוט שלבים. אם מוגדר Ollama — משדרג את הניסוח ב-LLM.

## 6. שכבת Purple Team (bluered.py)

### Broker (מתווך) 🟢

מקבל את ממצאי הצוות האדום, ממפה כל ממצא לכלל הגנה מתוך DEFENSE\_KB, מאחד עדויות ומדרג לפי חומרה.

### Blue Team (צוות כחול) 🔵

בסיס ידע של ~15 כללי הגנה. לכל איום: תיאור, פעולות הגנה/הקשחה, זיהוי וניטור, תצורה לדוגמה, ומיפוי MITRE ATT&CK (למשל T1190, T1110).

### Learning (למידה) 🧠

כל ממצא מנורמל לחתימה ונצבר ב-knowledge.json (מונה + first/last seen). כך המערכת "לומדת" אילו איומים חוזרים לאורך זמן.

### Orchestrator (מתזמר) 🟣

מנצח את כל הזרימה: מריץ את הצוות האדום, מפעיל את המתווך והכחול, מעדכן את הלמידה, ומפיק דוח Purple מלא.

## 7. זרימת נתונים — מאיפה הסוכנים יודעים?

שאלת מפתח: הסוכנים אינם "ממציאים" נתונים. הם שואבים משלושה מקורות ברורים וניתנים לביקורת.

| מקור                      | מה הוא מכיל                                                                                                                              | מי משתמש בו                  |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| 1. ידע סטטי (קוד/JSON)    | <code>tools.json</code> (קטלוג ופקודות), <code>agents.py</code> + Playbooks)<br>דפוסי חילוץ, <code>bluered.py</code> (כללי הגנה + MITRE) | Planner, Verifier<br>Blue    |
| 2. נתונים אמיתיים (הכלים) | הפלט האמיתי של כלי Kali על המטרה — פורטים, גרסאות, חולשות. <b>אמת הקרקע.</b>                                                             | → Executor<br>Verifier       |
| 3. זיכרון מצטבר (קבצים)   | <code>knowledge.json</code> (למידה), <code>activity.json</code> (יומן),<br><code>reports/</code> (דוחות)                                 | Learning<br>Dashboard, Brain |

```
intent + target
|
v
[Planner] — reads tools.json + PLAYBOOKS → plan (steps)
|
v
[Executor] — subprocess → REAL Kali tool → raw output
|
v
[Verifier] — regex patterns → findings
|
v
[Broker] — maps to DEFENSE_KB → threats + defenses
|
v
[Learning] — writes → knowledge.json (grows every run)
|
v
[Reporter] → report.md → reports/ + activity.json
```

איור 2 — זרימת הנתונים בין הסוכנים. הממצאים האמיתיים מגיעים תמיד מהכלים עצמם.

## 8. שכבת ה-Frontend ותצוגת המוח

אפליקציית עמוד יחיד (SPA) ב-vanilla JS ללא שלב build. שלושה מצבים: כלים, עוזר AI, ומרכז בקרה. הדשבורד מציג את מצב הסוכנים החי (נגזר מהרצות פעילות), מודיעין נצבר ויומן פעילות.

### 8.1 תצוגת "מוח סוכן"

לחיצה על סוכן פותחת ויזואליזציה עתידנית מבוססת SVG: ליבה זוהרת במרכז, יכולות במסלולים סביבה, סינפסות עם חלקיקי אנרגיה, וליבה "נושמת". המנגנון המרכזי הוא **Semantic Zoom (Level-of-Detail)**: ככל שמתקרבים (גלגלת) נחשפות שכבות עמוקות יותר — יכולות → כלים → פרטים. הנתונים נשאבים מ-`brain.json` ומועשרים בנתונים חיים (כלים מותקנים, ממצאים ידועים) דרך `/api/brain`.

## 9. אבטחה

- **בניית פקודות בטוחה:** מערך `argv` ללא `shell` — מונע הזרקת פקודות.
- **האזנה מקומית:** ברירת מחדל `127.0.0.1` בלבד — התהליך (`root`) אינו חשוף לרשת.
- **אימות אופציונלי:** טוקן משותף (`KALIGUI_TOKEN`) כהגנה נוספת.
- **מניעת traversal:** הגשת קבצים סטטיים מוגבלת לתיקיית `web` בלבד.
- **שלב אישור:** שכבת ה-AI מציעה תוכנית; ההרצה מתבצעת רק לאחר אישור המשתמש.

## 10. פריסה ותפעול בייצור

המערכת מותקנת כשירות **systemd** בתוך Kali WSL (`kali-gui.service`) עם הפעלה אוטומטית והתאוששות מקריסה (`Restart=on-failure`). מתקין חד-פעמי (`Install.bat` / `setup.ps1`) מבצע: בדיקת דרישות, תיקון DNS, התקנת השירות, ורישום משימת Windows להעלאת WSL בהתחברות.

```
systemctl status kali-gui      # מצב
systemctl restart kali-gui     # הפעלה מחדש
journalctl -u kali-gui -n 50   # לוגים
```

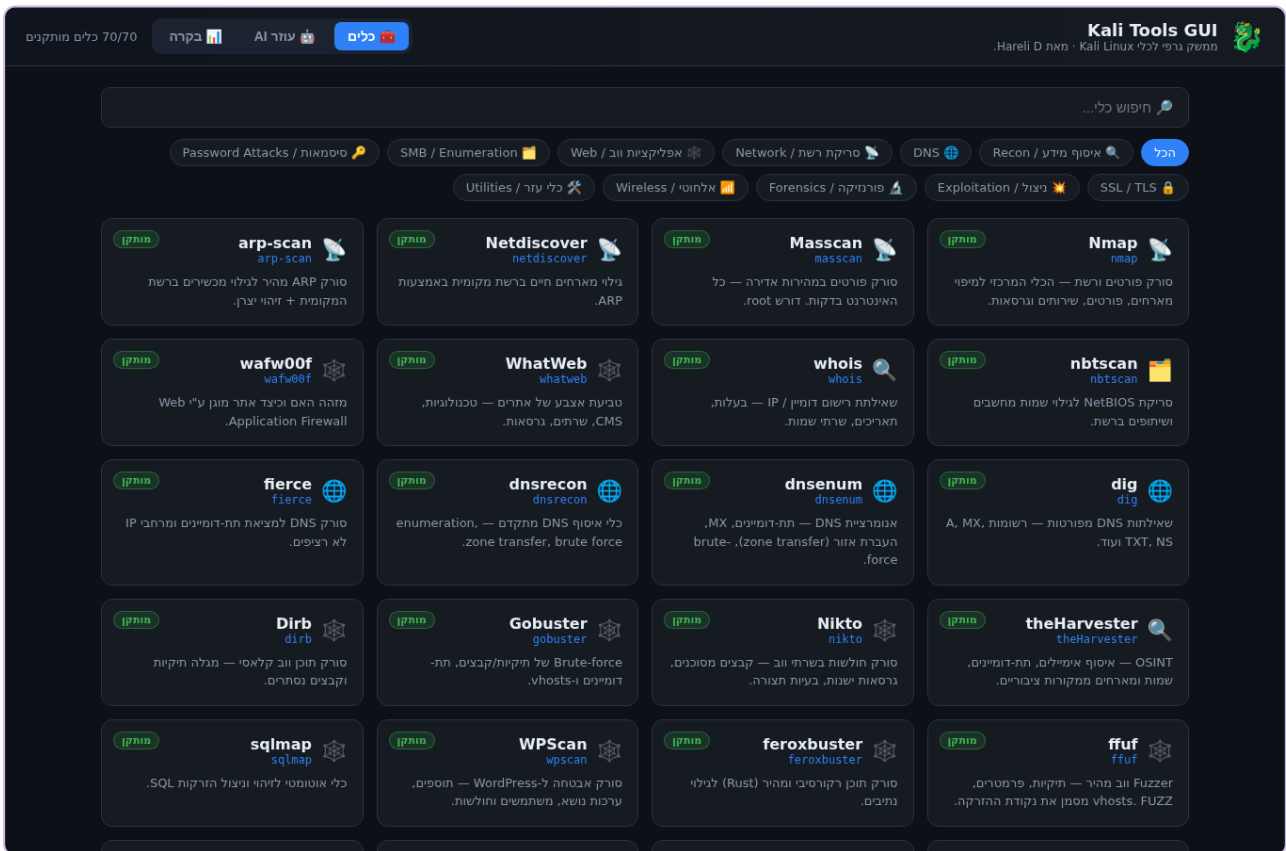
עמידות: טיפול גלובלי בשגיאות בכל בקשה, גיזום היסטוריית הרצות בזיכרון, לוגים לקובץ `journald`-ול, ושמירת דוחות לדיסק (שורדים אתחול).

## 11. טכנולוגיות (Tech Stack)

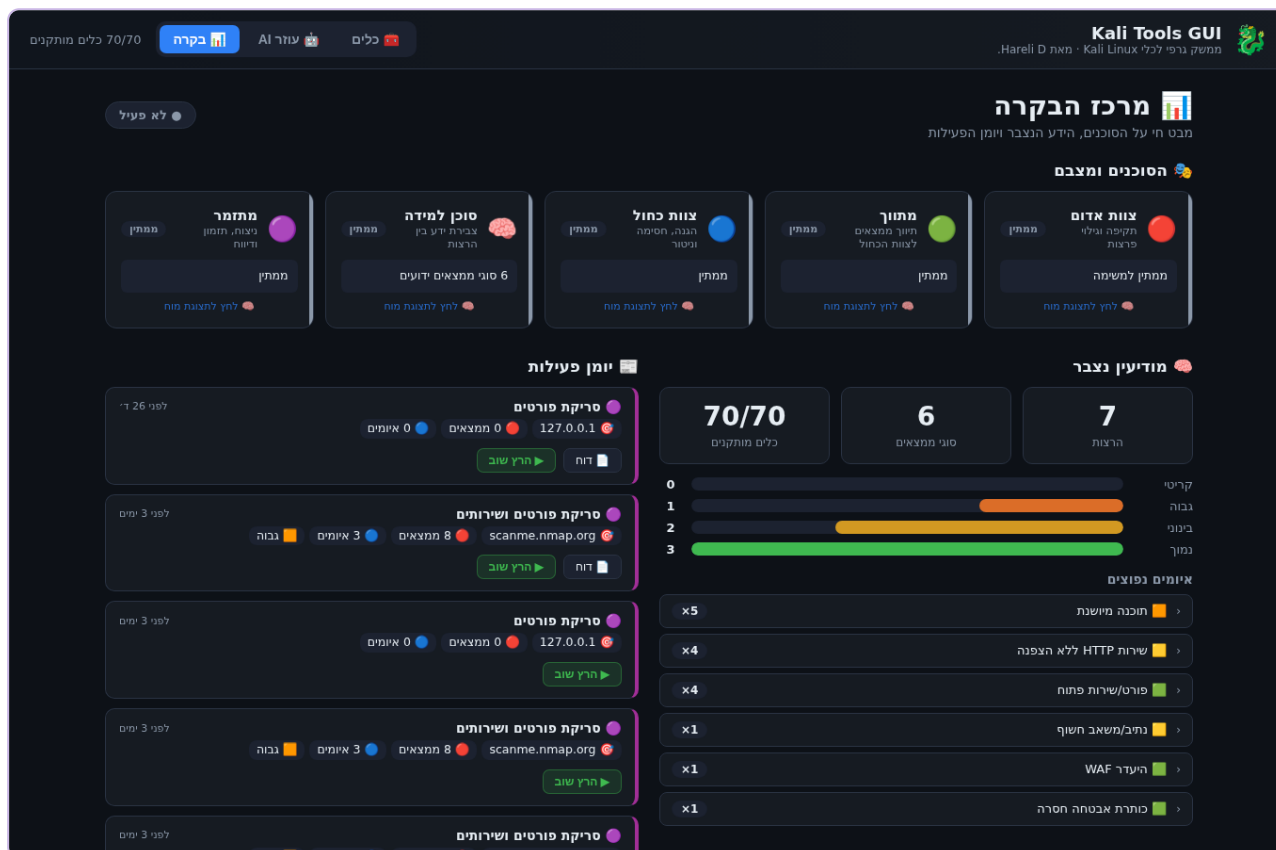
| שכבה        | טכנולוגיה                        | נימוק                                 |
|-------------|----------------------------------|---------------------------------------|
| Backend     | Python 3 — ספרייה סטנדרטית בלבד  | אפס תלויות; עובד על Kali מינימלי      |
| Frontend    | HTML/CSS/Vanilla JS (ללא build)  | פשטות, אפס כלי בנייה, RTL             |
| ויזואליזציה | SVG + requestAnimationFrame      | גרפיקה וקטורית אינטראקטיבית + אנימציה |
| אחסון       | קבצי JSON מקומיים                | שקיפות, פשטות, ניתן לביקורת           |
| סביבת ריצה  | Kali Linux תחת WSL2              | גישה מלאה לכלי הפנטסט                 |
| שירות       | systemd + Windows Task Scheduler | זמינות תמידית                         |



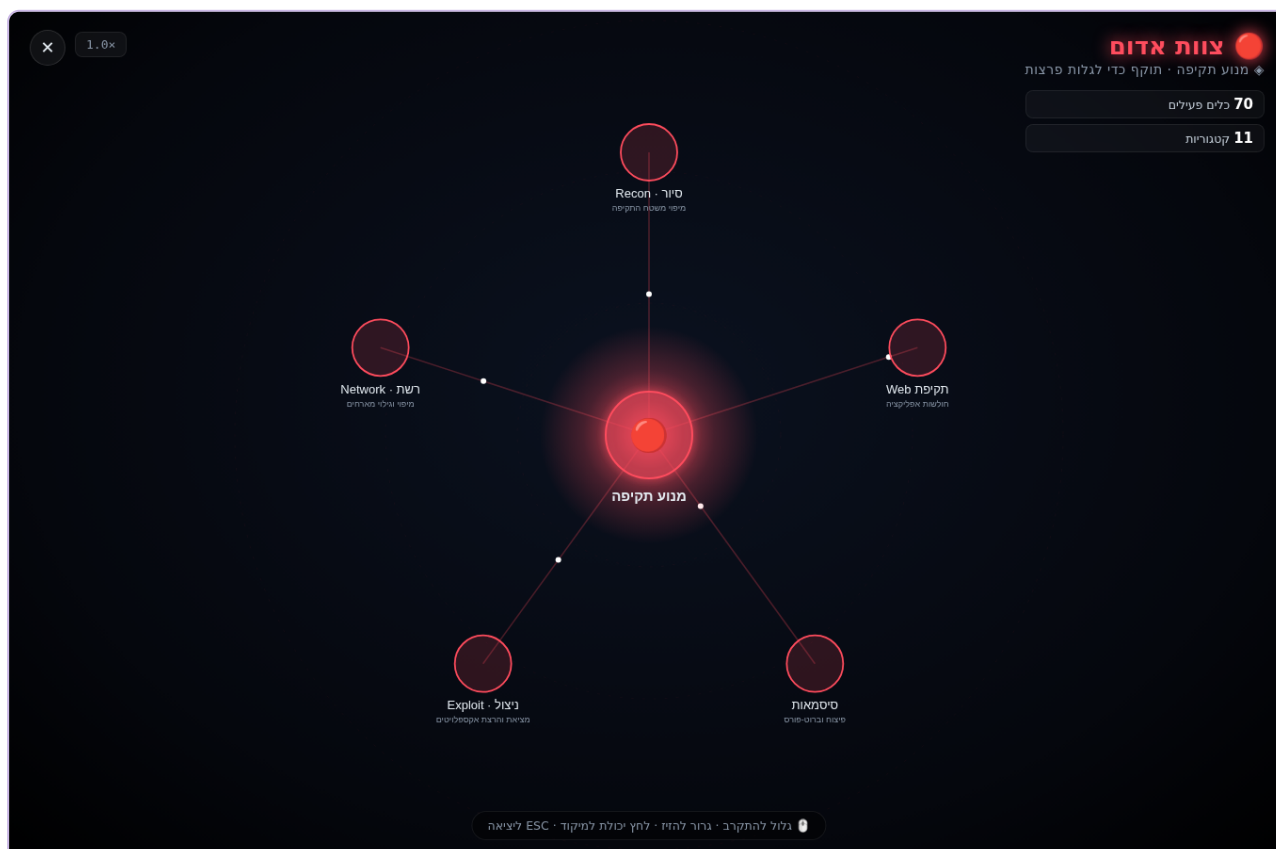
## 12. צילומי מסך מהמערכת



מסך 1 — מצב "כלים": קטלוג 70 הכלים עם חיפוש, סינון לפי קטגוריה וזיהוי מותקן.



מסך 2 — מרכז הבקרה: מצב הסוכנים החי, מודיעין נצבר (חומרה, איומים נפוצים) ויומן פעילות.



מסך 3 — תצוגת "מוח סוכן" (הצוות האדום): ליבה זוהרת, יכולות במסלולים, סינפסות ונתונים חיים.

המערכת מדגימה ארכיטקטורה מודולרית וברורה: מנוע כלים בטוח, שכבת סוכנים מבוססת-ידע דטרמיניסטית, גישת Purple Team שמחברת בין תקיפה להגנה, ולמידה מצטברת. כל רכיב שקוף וניתן להרחבה — הוספת כלי, Playbook או כלל הגנה נעשית בעריכת נתונים בלבד. התוצאה: כלי חינוכי ומעשי המנגיש בדיקות אבטחה ומחבר אותן לפעולה הגנתית.

— סוף מסמך התכן · הראלי דודאי · מכללת 2026 · HackerU —